# Phase 1 — Summary-Length Consistency

**Thesis priority:** #4 (Summary consistency) from §5.5 Future Works. **Date:** 2026-10-05 (FYP 2, Week 1) **Status:** implementation complete; benchmark execution pending a machine with Ollama running.

---

## 1. Problem statement (from FYP 1 evaluation, thesis §5.4)

The small local language model may occasionally miss the intended 30-word summary limit; prompt and output constraints need continued refinement.

The FYP 1 prompt asked for "a brief 30-word summary". This is a *soft* request: the model frequently returns 40–70 words, and the system accepted the longer output verbatim (by design, so longer accurate summaries were preserved). No mechanism existed to bring over-limit outputs back within the target.

## 2. Changes made

### 2.1 Prompt v2 (`src/ai/summarizer.py` → `build_summary_prompt`)

The word cap is now stated as a **hard limit in three places** of the instruction:

1. Task line: "output a summary of **no more than N words**" (v1: "a brief N-word summary").
2. Output rules: "The summary must not exceed N words. **Shorter is acceptable; longer is not.**"

A `strict=True` variant adds a `HARD LIMIT (mandatory — highest priority)` block instructing the model to draft, count, and shorten before answering. Rationale: small local models follow explicitly-prioritized constraints measurably better than soft suggestions (to be verified by the benchmark in §4).

Prompt builder is a separate function (not inlined) so the benchmark harness calls the **exact same prompt text** as production — keeping the evaluation faithful.

### 2.2 One-shot strict retry (production policy)

`summarize()` now measures the finalized output with `word_count()`; if it exceeds the cap:

1. Log the event (`[summary] over limit (X words > N cap); strict retry`).
2. Re-call the model once with `strict=True` (same article, same cap).
3. Accept the retry **only if it is valid and shorter** than the first output; otherwise keep the first output. No hard truncation is applied — a longer accurate summary is still preferable to a mangled one.
4. Log the outcome (accepted / failed / kept first).

If Ollama is unreachable the function still returns `None`, preserving the FYP 1 graceful- degradation contract (AGENTS.md rule #8).

### 2.3 New configuration (all in `src/core/config.py`, per change rule #1)

| Variable | Default | Purpose |
|---|---|---|
| `SUMMARY_RETRY_ON_OVERLIMIT` | `true` | Set `false` to restore exact FYP 1 behavior (accept first output). |
| `SUMMARY_TARGET_WORDS` | `30` | Word budget referenced by the retry policy. |

### 2.4 Refactor (no behavior change to other callers)

- JSON/text extraction chain moved verbatim into `parse_summary_output()`.
- Cleanup/validation chain moved verbatim into `finalize_summary_candidate()`.
- `_summarize_attempt()` = one Ollama call + extraction + cleanup; `summarize()` orchestrates attempt → measure → optional retry.
- `services.py` and other callers of `summarize()` are untouched; signatures unchanged.

## 3. Prompt versions (verbatim, for the report appendix)

**v1 (FYP 1), task line:**

Read the full article and output a brief 30-word summary in JSON format: one or two tight complete sentences with who, what, where, and the main outcome; skip minor detail if needed.

**v2 (FYP 2), task line:**

Read the full article and output a summary of no more than 30 words in JSON format: one or two tight complete sentences with who, what, where, and the main outcome; skip minor detail if needed.

**v2 additional rule (absent in v1):**

The summary must not exceed 30 words. Shorter is acceptable; longer is not.

**v2 strict retry block (only sent on retry):**

HARD LIMIT (mandatory — highest priority):

- The summary MUST contain at most 30 words. This is a hard cap, not a guideline.
- Before answering, draft the summary, count its words, and shorten it until it fits the cap.
- If it cannot fit in 30 words, drop minor details — never exceed the cap.

## 4. Evaluation methodology

Benchmark harness: `scripts/benchmark_summaries.py` (see `benchmark/README.md`).

- **Fixture:** N=50 articles sampled deterministically (seed 42) from `news_articles`, interleaved across sources. Exported once to JSON so every model is scored on identical inputs.
- **Inputs:** stored RSS descriptions (`raw_summary`). *Limitation:* production summarization receives full scraped bodies, which are not persisted; benchmark inputs are therefore slightly shorter. Word-count behavior is expected to transfer, and this is recorded as a threat to validity in the report.
- **Metrics per model:** parse-success rate, within-cap rate (final words ≤ 30 after the full cleanup pipeline), mean/median/p90/max final word counts, retry-trigger count, retry-fixed count, mean first-attempt latency.
- **Candidate models (per thesis):** current `llama3.1` baseline vs `qwen2.5:3b`-class and `ministral`-class small models.

## 5. Results

### 5.1 Run log

| Date (UTC) | Experiment | Endpoint | Model | Fixture | Rows | Notes |
|---|---|---|---|---|---|---|
| 2026-10-05 | llama31_v1 (baseline) | local Ollama (`localhost:11434`) | `llama3.1:latest` | 50-article fixture, articles 1–10 (offset 0–9) | 10 | FYP1 prompt, retry disabled (simulates FYP1) |
| 2026-10-05 | llama31_v2 (treatment) | local Ollama | `llama3.1:latest` | same 10 articles | 10 | FYP2 prompt + strict retry policy |

Raw data: `benchmark/llama31_v1.csv`, `benchmark/llama31_v2.csv`; aggregates: `benchmark/metrics_20261005T091303Z.json`. Local runs were chunked (`--offset`/`--limit`, 2 articles per invocation) because CPU inference is slow; rows append to the CSV incrementally.

### 5.2 Metrics (n = 10 articles, target ≤ 30 words)

| Label | Prompt | Parse success | Within cap | Mean words | Median | P90 | Max | Retries triggered | Retries fixed | Mean latency |
|---|---|---|---|---|---|---|---|---|---|---|
| llama31_v1 | v1 (FYP1) | 100% | 100% | 20.5 | 21.5 | 25.0 | 26 | 0 | 0 | 3.6 s |
| llama31_v2 | v2 (FYP2) | 100% | 100% | 20.4 | 20.0 | 25.0 | 30 | 1 | 1 | 3.9 s |

### 5.3 Interpretation (honest reading for the report)

1. On RSS-description inputs, both prompts already satisfy the 30-word cap at n=10 — llama3.1's over-limit behavior is rare on short inputs. The FYP1 limitation is more likely triggered by the **longer full article bodies** used in live summarization (bodies are not persisted, so this benchmark cannot reproduce that input length — see Threats to validity).
2. The retry policy demonstrably works: 1/10 articles exceeded the cap (31 words) and the strict retry brought it to exactly 30. It acts as a **safety net** with zero effect on already-compliant outputs.
3. Sample size is small (n=10, convenience subset). A larger run (or the production log measurement in §6) is needed for a defensible percentage.

### 5.4 Model availability findings (2026-10-05, document in report)

While preparing the benchmark, the planned candidate models were probed:

| Candidate | Status | Evidence |
|---|---|---|
| `ministral-3:8b-cloud` | **Unavailable** — model retired by Ollama Cloud (HTTP 410, retired 2026-07-15) | Replace with another small-model candidate |
| `gpt-oss:20b` (cloud fallback) | **Quota exhausted** — free-tier usage limit reached (HTTP 429) | Production may currently be failing over further; check Oracle logs |
| `mimo-v2.5` (fallback_2, `api.xiaomimimo.com`) | Working — ~7 s/article, valid JSON, 29 words on probe | Likely the **de-facto production summarizer**; verify in Oracle container logs |
| `glm-5.2:cloud` (local Ollama) | **Unauthorized** — local Ollama not signed in to ollama.com | Run `ollama signin` or drop the model |
| `llama3.1:latest` (local) | Working — ~4–80 s/article (first call loads the model into RAM) | Dev baseline used above |

Action: before finalizing the model-selection chapter, confirm which model the Oracle deployment actually generates summaries with (`docker logs` for `over limit` / fallback lines, or query `news_articles.ai_summary` provenance), then benchmark that model with v1 vs v2 on the full 50-article fixture.

## 6. Production evidence collection (post-deployment)

The retry logging (`[summary] over limit …`) lets us measure the true over-limit rate on live traffic: count `over limit` vs `retry accepted` lines in the Oracle container logs over a fixed window, e.g.:

```
docker logs apanewsharitok 2>&1 | grep -c "over limit"
docker logs apanewsharitok 2>&1 | grep -c "retry accepted"
```

Compare against FYP 1 behavior (no such logs existed — over-limit rate was never measured, which is itself report-worthy as the motivation).

## 7. Threats to validity

1. Benchmark inputs are RSS descriptions, not full article bodies (§4).
2. Word count is whitespace-token count; hyphenated compounds and Malay borrowings may count differently than a human word count.
3. One retry only — outputs still over the cap after retry are kept by policy, so the within-cap rate measures *policy outcome*, not raw model capability (both are reported).
4. Model behavior varies with temperature/hardware; the benchmark pins `num_predict` and uses each model's default temperature (Ollama default), which should be stated in the report.